

Go basics

Contents

1. [Program structure](#)
2. [Variables and constants](#)
3. [Control flow](#)
4. [Functions](#)
5. [Arrays](#)
6. [Maps](#)
7. [Strings](#)
8. [Structs](#)
9. [Interfaces \(basics\)](#)
10. [Errors](#)
11. [io basics](#)
12. [Visibility](#)
13. [Naming conventions](#)
14. [Go tooling](#)
15. [Build constraints](#)
16. [Best practices](#)
17. [Common gotchas](#)
18. [Quick reference](#)

The fundamentals that show up on day one of any Go codebase: program structure, types, control flow, functions, errors, and tooling. Topics already covered in their own sheets (goroutines, concurrency, slices, pointers, testing) are skipped here.

1. Program structure

Every Go file starts with a package declaration. The `main` package with a `main()` function is the entry point of an executable.

```
package main

import "fmt"

func main() {
    fmt.Println("hello")
}
```

Build and run:

```
go run .           # compile + run
go build           # produce binary
go build -o app .  # named output
./app
```

A directory is a package. All `.go` files in the same directory share the same package declaration. Imports are paths, not file names:

```
import (
    "fmt"
    "net/http"
    "github.com/foo/bar"
)
```

Unused imports and unused local variables are compile errors. Use `_` to keep an import for its side effects:

```
import _ "github.com/lib/pq"    // registers a driver
```

Modules

Every project lives in a module. `go.mod` declares the module path and dependencies.

```
go mod init github.com/me/proj    # create go.mod
go mod tidy                       # add missing, remove unused
go mod download                   # populate module cache
go get example.com/lib@v1.2.3     # add or upgrade
```

`go.sum` records cryptographic hashes of dependency contents. Commit both `go.mod` and `go.sum`.

2. Variables and constants

```
var x int = 10           // explicit type
var y = 10               // type inferred
var z int                // zero value (0)
a := 10                  // short declaration, only inside functions

const pi = 3.14          // typed at use site
const (
    name = "alice"
    age = 30
)
```

`:=` declares and assigns in one step. `=` only assigns. Mixing them works if at least one name on the left is new:

```
v, err := f()           // both new
v, err = f()            // both reassigned
v, err := f()           // v exists, err is new: valid (rare gotcha)
```

Zero values

Every type has a zero value. No “uninitialized” memory in Go.

Type	Zero value
numeric	<code>0</code>
bool	<code>false</code>
string	<code>""</code>
pointer, slice, map, chan, func, interface	<code>nil</code>
struct	each field is its zero value
array	each element is its zero value

This is why `var x int` is safe and useful. You can `x++` immediately.

iota

`iota` is a per-`const`-block counter that starts at 0 and increments on each line.

```
const (
    Sunday = iota    // 0
    Monday           // 1
    Tuesday          // 2
)

const (
    KB = 1 << (10 * (iota + 1)) // 1024
    MB                               // 1048576
    GB                               // 1073741824
)
```

Skip values with `_`. Reset by starting a new const block.

Basic types

Type	Notes
<code>int</code> , <code>int8</code> , <code>int16</code> , <code>int32</code> , <code>int64</code>	<code>int</code> is 32 or 64 bits depending on platform

Type	Notes
<code>uint</code> , <code>uint8</code> , <code>uint16</code> , <code>uint32</code> , <code>uint64</code> , <code>uintptr</code>	unsigned
<code>byte</code>	alias for <code>uint8</code>
<code>rune</code>	alias for <code>int32</code> , holds a Unicode code point
<code>float32</code> , <code>float64</code>	use <code>float64</code> unless you have a reason
<code>complex64</code> , <code>complex128</code>	rare in production code
<code>bool</code>	<code>true</code> / <code>false</code>
<code>string</code>	immutable byte sequence, UTF-8 by convention

Type conversions

Go has no implicit numeric conversion. Every cross-type assignment needs an explicit conversion.

```
var i int = 10
var f float64 = float64(i)
var u uint = uint(f)
```

This is verbose but catches a class of bugs other languages let through. String <-> byte/rune conversions:

```
b := []byte("hello")           // string to bytes (copy)
s := string(b)                 // bytes to string (copy)
r := []rune("héllö")           // string to runes (handles multi-byte)
s2 := string(r)                // runes to string
```

`string(65)` produces `"A"`, not `"65"`. Use `strconv.Itoa(65)` for the numeric conversion.

Operators

```
+ - * / %           // arithmetic
== != < <= > >=    // comparison
&& || !             // boolean
& | ^ &^ << >>      // bitwise (^ is XOR, &^ is AND NOT)
+= -= *= /= %=      // assignment ops
&= |= ^= <<= >>=    // bitwise assignment
```

`++` and `--` are statements, not expressions. `x = i++` is a compile error.

Integer division truncates toward zero. `7 / 2` is `3`. `%` has the sign of the dividend.

3. Control flow

if

```
if x > 0 {  
    // ...  
} else if x == 0 {  
    // ...  
} else {  
    // ...  
}  
  
// initializer form: scope is limited to the if/else block  
if err := f(); err != nil {  
    return err  
}
```

No parentheses around the condition. Braces are mandatory even for one-line bodies.

for

Go has one loop keyword. It covers C-style, while, and infinite loops.

```
for i := 0; i < 10; i++ { }    // C-style  
for x < 100 { }               // while  
for { }                       // infinite  
  
for i, v := range slice { }   // index, value  
for k, v := range mapVar { }  // key, value (random order)  
for i, r := range "hello" { } // byte index, rune  
for v := range channel { }    // until close  
for range n { }               // Go 1.22+: integer range
```

`break` and `continue` work as expected. Labels let you target outer loops:

```
outer:  
for i := 0; i < 10; i++ {  
    for j := 0; j < 10; j++ {  
        if cond { break outer }  
    }  
}
```

switch

No implicit fallthrough. Each case ends automatically. Use `fallthrough` if you need C behavior.

```

switch day {
case "mon", "tue", "wed":
    work()
case "sat", "sun":
    rest()
default:
    panic("unknown day")
}

// expression-less switch: replaces long if/else chains
switch {
case x < 0:
    neg()
case x == 0:
    zero()
default:
    pos()
}

// type switch (for interface values)
switch v := i.(type) {
case int:    fmt.Println("int", v)
case string: fmt.Println("string", v)
default:    fmt.Println("other")
}

```

Cases can have multiple values separated by commas. Cases can also have an initializer:

```

switch err := f(); err {
case nil:    success()
case io.EOF: done()
default:    log.Println(err)
}

```

4. Functions

```
func add(a, b int) int { return a + b }

// multiple return values
func divmod(a, b int) (int, int) { return a / b, a % b }

// named returns + naked return
func split(sum int) (x, y int) {
    x = sum * 4 / 9
    y = sum - x
    return                // returns x, y
}

// variadic
func sum(nums ...int) int {
    total := 0
    for _, n := range nums { total += n }
    return total
}
sum(1, 2, 3)
sum(slice...)           // spread a slice
```

First-class functions

Functions are values. Pass them, store them, return them.

```
add := func(a, b int) int { return a + b }
op := add
op(2, 3)

func apply(f func(int) int, x int) int { return f(x) }
```

Closures

A function literal captures variables from the enclosing scope by reference.

```
func counter() func() int {
    n := 0
    return func() int { n++; return n }
}

c := counter()
c() // 1
c() // 2
```

Closures are how Go does callbacks, option functions, and partial application. They keep their captured variables alive past the enclosing function's return.

defer

`defer` schedules a function call to run when the surrounding function returns (normally or via panic).

```
func read(path string) error {
    f, err := os.Open(path)
    if err != nil { return err }
    defer f.Close()
    // ... use f ...
    return nil
}
```

Defer rules

1. LIFO order. Multiple defers run in reverse declaration order.
2. Arguments are evaluated immediately, the call runs later.

```
x := 1
defer fmt.Println(x)    // prints 1, even if x changes later
x = 2
```

To capture by reference, wrap in a closure:

```
defer func() { fmt.Println(x) }()    // prints whatever x is at return
```

3. Defer in a loop accumulates. Don't defer `f.Close()` inside a long loop. Break out into a helper.
4. Deferred functions see and can mutate named returns.

```
func f() (n int) {
    defer func() { n++ }()
    return 1    // returns 2
}
```

Cost: a few nanoseconds per defer. Negligible outside extremely hot paths.

5. Arrays

Fixed-length, value semantics. Rarely used directly; slices are usually what you want.

```

var a [3]int           // [0 0 0]
b := [3]int{1, 2, 3}
c := [...]int{1, 2, 3} // length inferred
len(a)                // 3

a[0] = 10
for i, v := range a { }

```

Arrays are values. Passing one to a function copies the whole thing. Same for assignment. This is the main reason slices exist.

6. Maps

A hash map. Keys must be comparable (`==` defined): numbers, strings, pointers, channels, interfaces, structs/arrays of comparables. Not slices, maps, or functions.

```

m := map[string]int{"a": 1, "b": 2}
m := make(map[string]int)
m := make(map[string]int, 100) // capacity hint

m["c"] = 3
v := m["x"] // zero value if missing (0 here)
v, ok := m["x"] // ok is false if missing
delete(m, "a")
len(m)
for k, v := range m { } // random order, every time

```

Map gotchas

- Nil map writes panic. `var m map[string]int; m["a"] = 1` blows up. Reads on a nil map return the zero value (safe).
- Iteration order is randomized intentionally to discourage relying on it.
- Concurrent map access is not safe. Even one writer + concurrent readers is a race. Use `sync.RWMutex` or `sync.Map`.
- You can't take the address of a map element. `&m["a"]` is a compile error. Read out into a variable, modify, write back. Or use a `map[K]*V`.
- Struct field assignment through a map needs the value out first.

```

m := map[string]Point{"a": {1, 2}}
m["a"].X = 5 // compile error: assignment to struct field in map
p := m["a"]; p.X = 5; m["a"] = p // works

```

7. Strings

Strings are immutable byte sequences. They're not character arrays.

```
s := "héllö"
len(s)           // 6 (bytes, not characters)
s[0]             // byte 0x68 ('h')
s[1]             // byte 0xc3 (start of multi-byte é)
```

Iterating correctly

`for i, r := range s` decodes UTF-8 and gives you runes:

```
for i, r := range "héllö" {
    fmt.Printf("%d %c\n", i, r)
}
// 0 h
// 1 é
// 3 l    <- index jumps from 1 to 3 (é took 2 bytes)
// 4 l
// 5 o
```

Common operations

```
import "strings"

strings.Contains(s, "ll")
strings.HasPrefix(s, "he")
strings.Split("a,b,c", ",")           // ["a" "b" "c"]
strings.Join([]string{"a","b"}, "-") // "a-b"
strings.ToLower(s)
strings.TrimSpace(s)
strings.Replace(s, "old", "new", -1) // -1 means all
strings.ReplaceAll(s, "x", "y")      // shortcut
```

Building strings

Concatenation with `+` allocates a new string every time. For loops, use `strings.Builder`:

```
var b strings.Builder
for _, w := range words {
    b.WriteString(w)
    b.WriteByte(' ')
}
result := b.String()
```

For one-off formatting, `fmt.Sprintf` is fine.

8. Structs

A typed collection of fields.

```
type Point struct {
    X, Y int
}

p := Point{X: 1, Y: 2}           // named fields (recommended)
p := Point{1, 2}                 // positional (brittle; field order matters)
p := Point{}                     // zero value: {0, 0}
p := &Point{X: 1}                // pointer literal

p.X = 10
```

Embedded fields (composition)

```
type Animal struct {
    Name string
}

func (a Animal) Greet() { fmt.Println("hi from", a.Name) }

type Dog struct {
    Animal           // embedded, not named
    Breed string
}

d := Dog{Animal: Animal{Name: "Rex"}, Breed: "Lab"}
d.Greet()           // promoted from Animal
d.Name              // also promoted
```

Embedding promotes fields and methods. It's composition, not inheritance. There's no polymorphism through embedding; the outer type doesn't override the inner's methods unless explicitly redefined.

Tags

Struct tags are string metadata read at runtime by encoders.

```
type User struct {
    ID    int    `json:"id"`
    Email string `json:"email,omitempty" validate:"email"`
}
```

`encoding/json`, `encoding/xml`, `database/sql`, and most validators use tags.

9. Interfaces (basics)

An interface is a set of method signatures. A type satisfies an interface implicitly by implementing all its methods. There's no `implements` keyword.

```
type Stringer interface {
    String() string
}

type User struct{ Name string }
func (u User) String() string { return u.Name }

var s Stringer = User{Name: "alice"}           // works because User has String()
```

The empty interface and any

`interface{}` (or its alias `any` from Go 1.18) accepts any type. Use for generic containers, formatting, JSON decoding. Reach for generics first if the types are known at compile time.

```
func print(v any) { fmt.Println(v) }
```

Type assertions

```
var i any = "hello"
s := i.(string)           // panics if i isn't a string
s, ok := i.(string)       // safe form: ok is false if wrong type
```

For multi-way dispatch, use a type switch (see Control flow).

Common standard library interfaces

Interface	Method	Used for
<code>io.Reader</code>	<code>Read([]byte) (int, error)</code>	streaming input
<code>io.Writer</code>	<code>Write([]byte) (int, error)</code>	streaming output
<code>io.Closer</code>	<code>Close() error</code>	resources that need cleanup
<code>error</code>	<code>Error() string</code>	errors
<code>fmt.Stringer</code>	<code>String() string</code>	custom print format
<code>sort.Interface</code>	<code>Len</code> , <code>Less</code> , <code>Swap</code>	sortable collections (legacy; prefer <code>slices.SortFunc</code>)

Build small interfaces. The standard library's interfaces are 1-3 methods. The Go proverb: "the bigger the interface, the weaker the abstraction."

10. Errors

Errors are values. The `error` interface has one method:

```
type error interface {  
    Error() string  
}
```

Functions return errors as the last return value. The caller checks immediately.

```
f, err := os.Open(path)  
if err != nil {  
    return fmt.Errorf("open %s: %w", path, err)  
}  
defer f.Close()
```

Creating errors

```
errors.New("not found")  
fmt.Errorf("user %d: %w", id, err)           // %w wraps the underlying error
```

Sentinel errors

Predefined error values for direct comparison.

```
var ErrNotFound = errors.New("not found")  
  
if err == ErrNotFound { /* handle */ }  
if errors.Is(err, ErrNotFound) { /* handle (works through wrapping) */ }
```

Typed errors

```
type ValidationError struct {  
    Field string  
    Msg    string  
}  
  
func (e *ValidationError) Error() string {  
    return fmt.Sprintf("%s: %s", e.Field, e.Msg)  
}  
  
var verr *ValidationError  
if errors.As(err, &verr) {  
    fmt.Println(verr.Field)  
}
```

errors.Is vs errors.As

	Use for
<code>errors.Is(err, target)</code>	exact sentinel match (through wrapping)
<code>errors.As(err, &target)</code>	extracting a typed error (through wrapping)

Panic and recover

`panic` halts normal execution and unwinds the stack. `recover` (only inside a deferred function) catches it.

```
defer func() {  
    if r := recover(); r != nil {  
        log.Println("recovered:", r)  
    }  
}()  

```

Use `panic` for truly unrecoverable bugs (nil deref of a required dependency at startup, “this should never happen” invariants). Don’t use `panic` for normal error handling. Don’t use `recover` to make `panic` into a try/catch.

11. io basics

`io.Reader` and `io.Writer` are the central streaming abstractions.

```
type Reader interface { Read(p []byte) (n int, err error) }  
type Writer interface { Write(p []byte) (n int, err error) }
```

Anything that produces bytes (file, network, in-memory buffer, gzip stream) implements `Reader`. Anything that consumes bytes implements `Writer`. The stdlib chains them:

```
io.Copy(dst, src) // pipe one to the other
buf := new(bytes.Buffer)
io.Copy(buf, resp.Body)
data, err := io.ReadAll(resp.Body) // simple cases
```

`bytes.Buffer` and `strings.Reader` adapt in-memory data to the interfaces. `bufio.Scanner` wraps a `Reader` for line-by-line reading:

```
scanner := bufio.NewScanner(file)
for scanner.Scan() {
    fmt.Println(scanner.Text())
}
if err := scanner.Err(); err != nil { /* handle */ }
```

12. Visibility

A name starting with an uppercase letter is exported (visible outside the package). Lowercase is package-private.

```
package mypkg

func Public() {} // visible elsewhere
func private() {} // only inside mypkg

type Thing struct {
    Name string // exported field
    secret string // package-private field
}
```

The rule applies to every name: functions, types, fields, constants, methods.

13. Naming conventions

- Package names: short, lowercase, no underscores (`bytes`, `http`, `strconv`). Never `myUtils`.
- Variable names: short for short scopes (`i`, `r`, `buf`), longer for wider scopes.
- Receivers: 1-2 letters tied to the type (`func (u *User)`, `func (s *Server)`). Not `this` or `self`.
- Errors start with lowercase: `errors.New("not found")`, not `"Not found"`.
- Interfaces end in `-er` if they're single-method (`Reader`, `Writer`, `Stringer`).
- Constants use MixedCaps, not UPPER_SNAKE.
- Acronyms keep their case: `HTTPServer`, `userID`, `parseURL`.

14. Go tooling

Command	What it does
<code>go run .</code>	compile and run the current package
<code>go build</code>	compile to a binary in the current dir
<code>go install</code>	compile and put binary in <code>\$GOBIN</code>
<code>go test ./...</code>	run all tests recursively
<code>go test -run TestX -v</code>	run a specific test verbosely
<code>go test -race ./...</code>	run with the race detector
<code>go test -cover ./...</code>	print coverage percentage
<code>go vet ./...</code>	static checks for common mistakes
<code>go fmt ./...</code>	format source (or <code>gofmt -w</code>)
<code>goimports -w .</code>	format + manage imports
<code>go mod tidy</code>	clean up go.mod
<code>go doc fmt.Println</code>	show docs for a symbol
<code>go env</code>	show Go's environment
<code>go generate ./...</code>	run <code>//go:generate</code> directives

`go fmt` is non-negotiable. There's one canonical formatting and the tool enforces it. Most IDEs run it on save.

15. Build constraints

Conditional compilation by filename suffix or build tag.

```
// File: file_linux.go           - only built on Linux
// File: file_amd64.go          - only built on amd64
// File: file_linux_amd64.go    - both

//go:build integration           - build tag at top of file (above package)
// +build integration            - old style, still seen
```

Run tagged builds with `go test -tags integration`.

16. Best practices

1. Run `gofmt`, `go vet`, and `go test -race` on every change. Make this CI.
2. Handle every error. Don't `_ = err`. If you genuinely don't care, comment why.

3. Wrap errors with `%w` and context. `fmt.Errorf("user %d: %w", id, err)`.
4. Return errors, don't panic. Reserve panic for unrecoverable startup invariants.
5. Accept interfaces, return concrete types. Functions take small interfaces; they return real structs.
6. Keep packages cohesive. A package should do one thing. Split when it grows two reasons to change.
7. Document exported names. A comment that starts with the name and ends with a period is the convention.
8. Avoid `init()`. It runs at import time, hides side effects, and complicates testing.
9. Don't fight `go fmt`. Even if you disagree with a choice, the consistency is worth more.
10. Run with the latest minor version. Go is backwards-compatible within a major; upgrade often.

17. Common gotchas

Gotcha	Symptom	Fix
<code>:=</code> shadows outer variable	"declared but not used" or stale value	Use <code>=</code> once the name exists
Unused import / variable	Compile error	Delete it, or <code>_ = it</code>
Nil map write	Panic	<code>make(map[K]V)</code> before writing
<code>&m["key"]</code>	Compile error	Pull value into a var, or use <code>map[K]*V</code>
Iterating a map for stable order	Different order each run	Sort the keys first
Reading a string by index expecting char	Get a byte	Range over string for runes
<code>defer f()</code> inside long loop	File handles pile up	Refactor into a helper function
Capturing loop variable in goroutine (Go < 1.22)	All goroutines see same value	<code>i := i</code> shadow, or pass as arg
Comparing interfaces to typed nil	<code>!= nil</code> returns true unexpectedly	Use concrete type for the variable
<code>int(myFloat)</code> truncates	Lost decimals silently	<code>math.Round</code> first if you need rounding
Reading from a closed channel	Returns zero value, no panic	Check <code>v, ok := <-ch</code>
<code>fmt.Println(err)</code> on nil	Prints ""	Check <code>err != nil</code> first

18. Quick reference

What	Syntax
Declare and infer	<code>x := 10</code>
Constant	<code>const pi = 3.14</code>
Enum with iota	<code>const (A = iota; B; C)</code>
Type conversion	<code>float64(x)</code>
Multi-return	<code>func f() (int, error)</code>
Named return	<code>func f() (n int, err error)</code>
Variadic	<code>func f(xs ...int)</code>
Spread	<code>f(slice...)</code>
Deferred call	<code>defer f()</code>
Range over slice	<code>for i, v := range s</code>
Range integer (Go 1.22+)	<code>for i := range n</code>
Range over map	<code>for k, v := range m</code>
Type assertion	<code>s := i.(string)</code>
Type switch	<code>switch v := i.(type)</code>
Wrap error	<code>fmt.Errorf("...: %w", err)</code>
Compare error	<code>errors.Is(err, ErrX)</code>
Extract typed error	<code>errors.As(err, &target)</code>
Struct tag	<code>`json:"name"`</code>
Build constraint	<code>//go:build linux</code>
Format source	<code>go fmt ./...</code>
Tidy modules	<code>go mod tidy</code>